

MTP-Duplex: Attaching Full-Duplex Turn-Taking to Any LLM with an MTP-Style Protocol Companion

Penhfel et al.

private project — custom-duplex-cascade-pt-br/mtp_like

Abstract—Full-duplex speech dialogue requires the conversational model to perform two tasks at once: generating answer content *and* controlling turn-taking (when to speak, when to stop, when to acknowledge). The DuplexCascade approach trains these two skills into a single model by interleaving answer text with control tokens. We argue this coupling is unnecessary: a small, dedicated *protocol companion* — fine-tuned on the same duplex micro-turn data — can act as a MultiToken-Prediction-style (MTP) head for the control stream, running alongside an otherwise untouched big model. The companion predicts, at every conversation event, only the next duplex control tag (or nothing), while the big model only ever writes answer text. We train a 0.8B companion (Qwen3.5-0.8B) on the DuplexCascade pt-BR duplex dataset (10k dialogues) augmented with *barge-in* and *silence-semantics* supervision, and demonstrate: (i) 97.5% tag-stream accuracy with perfect F1 on the critical controller decisions (barge-in interruption and backchannel recognition), (ii) correct controller behavior on hand-built conversation probes, and (iii) a working end-to-end browser demo where a stock Qwen3-4B model engages in barge-in-capable full-duplex conversation, driven entirely by the companion. The approach is model-agnostic: turn-taking can be attached to any LLM without fine-tuning it.

Index Terms—full-duplex dialogue, turn-taking, spoken dialogue systems, MTP, VAD-free, companion model

I. INTRODUCTION

SPOKEN dialogue systems are moving from turn-based (“push-to-talk”) interaction to *full-duplex* conversation, where both parties may speak at any time. The DuplexCascade paper [1] established that a language model can learn the full-duplex protocol directly: its training data interleaves user and assistant *micro-turns* and supervises a set of control tokens (“user is speaking”, “user finish speaking”, “user interruption”, “user backchannel”, “user is thinking”, “system backchannel”) so that the same model that writes the answer also emits the control stream. The model must therefore be fine-tuned — deeply — for the turn-taking skill, and the skill is locked into that specific checkpoint.

This paper proposes a separation of concerns. Instead of teaching the answer model the protocol, we train a *small companion model* that predicts *only* the protocol stream, on behalf of the bigger model, while the bigger model continues to do what it already does: produce answer text. This is the same division of labor as MultiToken Prediction (MTP) heads (e.g., DeepSeek [2]), where a lightweight module predicts tokens one step ahead alongside the main model — except that our companion predicts a different vocabulary: the *duplex cascade protocol*, not the next answer token. On positions where no control token applies it emits nothing (an empty

string that never enters the context). The companion therefore acts as the *controller* of the system: it decides turn starts and ends, barge-ins, backchannels and silences, without any voice-activity detection (VAD).

Concretely we contribute:

- a training scheme that reuses the exact DuplexCascade pt-BR duplex dataset (same micro-turns, same phenomena) but with *tag-only* supervision at item boundaries, plus two controller-focused augmentations: *barge-in* records (user words arriving over an open assistant utterance → interruption) and *silence-semantics* records (user utterance + silence → “finish speaking”), including minimal single-utterance variants;
- a 0.8B companion (Qwen3.5-0.8B, text-only) fine-tuned with unsloth QLoRA on a single RTX 4080 in about two hours, reaching 97.5% tag-stream accuracy and perfect F1 on the interruption and backchannel decisions;
- a runtime *orchestrator* that consumes ASR word events and silence, queries the companion, and drives any stock LLM — no VAD, no special tokens in the big model’s vocabulary;
- an end-to-end browser demo in which a stock Qwen3-4B model holds a barge-in-capable full-duplex conversation in Brazilian Portuguese, entirely controlled by the companion.

II. BACKGROUND AND RELATED WORK

A. DuplexCascade

The DuplexCascade paper [1] constructs a full-duplex speech dialogue pipeline by (1) converting plain dialogues into interleaved micro-turns: user turns are split into 1–7-token chunks, each answered by a system tag such as “user is speaking”; the final chunk is answered by “user finish speaking” plus the first answer chunk; system turns are delivered in 10-token bursts separated by “no voice” user turns; and six interaction phenomena (pauses, interruptions, backchannels, thinking pauses, etc.) are simulated probabilistically; and (2) fine-tuning an LLM on the resulting stream with masked, weighted cross-entropy so that it learns to interleave answer text and control tokens. The model’s own outputs therefore *are* the control stream, and the pipeline (ASR → LLM → TTS) reacts to them.

The key limitation we address: answer quality and turn-taking skill are co-trained into one set of weights. Attaching the protocol to a new or better LLM requires a full re-fine-

tune of that model with protocol tokens in its vocabulary, and the model cannot be used with arbitrary serving stacks.

B. MultiToken Prediction

DeepSeek-V3 [2] introduces MTP modules: each MTP layer takes the previous layer’s hidden state and the *next token’s embedding* and predicts the token after next, in parallel with the main model’s next-token prediction. The main model’s task is unchanged; the MTP head just looks ahead. Our companion is analogous in *role* but different in subject: it looks ahead in the *protocol stream* — it predicts which duplex control token fires at the next boundary — while the main model continues to predict answer tokens. The big model is completely untouched.

C. Turn-taking without VAD

VAD-free turn-taking has been explored via “word events and silence timers” (e.g., in streaming ASR pipelines). DuplexCascade itself is VAD-free at the LLM level: “no voice” markers replace VAD, and the model learns their meaning. Our companion learns the same semantics from the same data, but as a standalone decision module: it consumes the ASR word stream and the assistant’s generated text and decides the protocol state. This keeps the big model’s serving stack completely standard.

III. METHOD

A. The protocol vocabulary and the companion’s role

Let $\mathcal{T} = \{\text{no_tag}, \text{user_is_speaking}, \text{user_finish_speaking}, \text{user_interrupt_marker}, \text{no_voice}, \text{no_tag_silence}, \text{no_tag_silence_end}, \text{no_tag_silence_start}, \text{no_tag_silence_end_marker}, \text{no_tag_silence_start_marker}, \text{no_tag_silence_end_marker_silence}, \text{no_tag_silence_start_marker_silence}\}$ be the companion’s output vocabulary, where `no_tag` is a label-only symbol (`<no tag>`) that never enters the context. At every conversation event the companion maps the duplex context (the interleaved user/assistant item stream, rendered in ChatML) to a distribution over $\mathcal{T}$; a tag fires only if its probability exceeds a per-tag threshold. Fired tags are appended to the context as assistant items, exactly like the training stream; `no_tag` means “emit nothing”.

The last two tags implement the *promptable controller*: when a system instruction (a *rule*) is present in the context, the companion can fire `system_take_floor` (the system interrupts the user now — an instant, sub-second response) or `system_handover` (control goes to the big model, which decides what to do, possibly nothing).

B. Training targets from the DuplexCascade data

We reuse the pt-BR duplex dataset produced by the companion project (10k dialogues; the same 50k-source micro-turn pipeline as [1], with variable-length system chunks). For every item i of a dialogue we define the target as the tag carried by the *next* assistant item:

$$\text{target}(i) = \begin{cases} \text{special}(\text{item}_{i+1}) & \text{item}_i \text{ is user} \\ \text{no_tag} & \text{item}_i \text{ is assistant} \end{cases} \quad (1)$$

where `special` is the control token of the following assistant micro-turn (or `no_tag` when the micro-turn is plain text).

Each dialogue becomes one training sequence with masked labels: cross-entropy is applied only at item boundaries (the final token of each item). All other positions are -100 .

This single transformation gives the companion exactly the supervision the paper’s model received, but as a *classification* of the protocol state instead of a generation task.

C. Controller augmentations

The vanilla targets cover the paper’s phenomena, but two behaviors are critical for a controller and under-represented:

a) *Streaming-aware supervision (SAS)*.: The runtime queries the companion while user words are still arriving, so for every multi-token user chunk we add partial-prefix records labeled `user_is_speaking` (words still arriving), except partials of an interrupting chunk, which are labeled `user_interruption`.

b) *Barge-in (controller) records*.: At random points inside every system turn we render the current assistant item as an *open* item (partial content, no closing token — speech in progress) followed by (a) the user’s next real words $\rightarrow$ `user_interruption` (plus a partial-prefix variant: the first words of the barge-in), (b) `<no voice>` $\rightarrow$ `no_tag` (silence during speech means keep talking), and (c) a sampled pt-BR backchannel word $\rightarrow$ `user_backchannel` (an acknowledgment is not a barge-in). This is what lets the companion distinguish “the user barged in” from “the user just said aham” from “the user paused”.

c) *Silence-semantics (turn-end) records*.: The runtime inserts `<no voice>` markers on ASR silence, so the data must define what silence means in every state: non-final chunk + silence $\rightarrow$ `speaking` (paused mid-speech); *final* chunk + silence $\rightarrow$ `finish speaking` (turn complete — the answer must start; the original data never shows this, since `finish` always directly follows the final chunk); partial final chunk + silence $\rightarrow$ `speaking`; and minimal single-utterance records [`u chunk`, `<no voice>`] (weight 5) for short single-batch utterances, without which the model never fires `finish` on a two-item context.

d) *Promptable-controller (rule) records*.: To make the controller *promptable*, we render a system instruction (the rule) as the first context item and label the boundary after a trigger chunk with `system_take_floor` or `system_handover`: e.g. the CPF rule “No CPF, apenas números são permitidos. Se o cliente disser uma letra em vez de um número, interrompa...” with a letter chunk after a digit dictation $\rightarrow$ `take_floor`. The synthetic scenarios use the *real dictation shape* (digit chunks with commas, variable count and order, the full alphabet as letters) so the companion learns the *concept* (number vs. letter, rule trigger vs. unrelated words) instead of memorized words; negatives (rule present + non-trigger chunk $\rightarrow$ the normal tag) prevent spurious fires, and a held-out probe confirms that without a rule the system tags never fire. About half of the regular records additionally get a generic system instruction (“Gerencie a conversa normalmente...”) so the base controller behaviors survive the instruction-conditioned training.

D. Loss

We fine-tune with masked cross-entropy and per-token label weights (tags carry the paper’s relative weights, retuned for the 9-way decision: finish = 3, interruption = 6, backchannel = 3, thinking = 2, speaking = 2, take_floor = 5, handover = 3, no_tag = 0.5), plus a per-record multiplier (loss_weight) so the rare controller patterns dominate the gradient (silence negatives $\times 3$, silence-finish $\times 3$, barge-in positives $\times 1.5$, rule positives $\times 5$).

Because the companion must keep its *language* and instruction-following ability (the rule mechanism depends on it), the content positions use an SDFT-style distill term: forward KL from the frozen base model over the teacher’s top- k support ($k = 32$),

$$\mathcal{L} = \underbrace{\frac{1}{N_{\text{sup}}} \sum_{i \in \text{bounds}} w_i \text{CE}(\hat{p}_i, y_i)}_{\text{tag boundaries}} + \lambda \underbrace{\frac{1}{N_{\text{sup}}} \sum_{i \in \text{content}} D_{\text{KL}}(p_i^{\text{base}} \parallel \hat{p}_i)}_{\text{content (forward-KL)}}$$

with $\lambda = 0.3$. The teacher is the same weights with the adapters disabled (no second model, no extra VRAM); this preserves the base model’s language capability while the boundaries learn the protocol.

E. Inference

At runtime, the companion is queried per event (new ASR words, silence timer, assistant sentence boundary). Its decision is a distribution over $\mathcal{T}$; a tag fires only if its probability $\geq \theta_{\text{tag}}$ and, for `user_interruption`, only if the last user item is not the orchestrator-inserted `<no voice>` marker (silence must never hard-stop the assistant). The context rendering must match the training distribution exactly: user items are always closed (`<|im_end|>`), even for partial content; only an in-progress assistant item is rendered open; the rule, when configured, is the first system item.

F. Orchestrator

The orchestrator maintains the duplex context and reacts to events:

- user words arrive $\rightarrow$ query; `user_is_speaking` $\rightarrow$ keep waiting (chunk closes when the next words arrive); `user_finish_speaking` $\rightarrow$ close the user turn and start the big model; `user_interruption` $\rightarrow$ hard stop the answer and TTS;
- silence (word-gap timer, no VAD) $\rightarrow$ insert `<no voice>` and query; `finish` $\rightarrow$ start the answer; `no_tag` $\rightarrow$ keep talking; `thinking` $\rightarrow$ wait;
- assistant sentence boundary $\rightarrow$ query; `no_tag` $\rightarrow$ continue;
- `user_backchannel` $\rightarrow$ keep talking; `system_backchannel` $\rightarrow$ play a canned acknowledgment;
- `system_take_floor` $\rightarrow$ reset the user’s turn and respond *instantly* with a canned phrase over the local TTS (the big model’s 2–4 s generation would never cut the user off); the rest of the interrupted utterance is not re-fed (no double fire);

- `system_handover` $\rightarrow$ ask the big model to decide (respond now, or do nothing) with the rule and the user’s words.

The big model receives plain text only and is otherwise untouched — except that it now gets the *conversation history* (previous turns and its own answers, with protocol tags filtered out), which the companion derives from the duplex context; a stateless big model otherwise repeats itself (e.g. a bank assistant re-asking for a CPF it already requested).

IV. EXPERIMENTS

A. Setup

a) *Data.*: The 10k-dialogue pt-BR duplex dataset (sft pipeline, 2026). The prep builds $\sim 946\text{k}$ records: 10k main streams, 200k SAS, 155k barge-in, 553k silence-semantics, 28k prompt-rule scenarios, and the instruction mix. Split 99.5/0.5 with the same seed as the sft project.

b) *Companion.*: principled-intelligence/Qwen3.5-0.8B (the Qwen3.5-0.8B language model, 0.75B), QLoRA-style LoRA ($r=32$, $\alpha=32$) on all linear layers plus embedding/LM-head fine-tuning of the 10 new special tokens, bf16, unsloth 2026.9.7 (Qwen3.5 fast path with *fla/causal-conv1d* kernels), 3000 steps, effective batch 16 (per-device batch 1×16 accum — batch 2 exceeds the 16 GB VRAM of the host’s WSL2 setup), lr 10^{-4} , distill $\lambda = 0.3$, $k = 32$, $\sim 5\text{h}$ on a single RTX 4080.

c) *Big model.*: Stock Qwen/Qwen3-4B-Instruct-2507 for the end-to-end demo (see §V for why Qwen3.5-4B is not the default).

B. Tag-stream accuracy

Table I reports per-tag precision/recall/F1 on the held-out split (3,222 boundary decisions), computed with restricted argmax over $\mathcal{T}$.

TABLE I
PER-TAG RESULTS OF THE TRAINED COMPANION (V6) ON THE HELD-OUT DUPLEX SPLIT.

tag	P	R	F1
no tag (emit nothing)	1.00	0.95	0.973
user is speaking	0.99	0.92	0.953
user finish speaking	0.79	0.98	0.877
user interruption	0.95	0.99	0.969
user backchannel	1.00	0.98	0.99
user is thinking	0.90	1.00	0.949
system take floor	1.00	1.00	1.00
system handover	1.00	1.00	1.00
accuracy	0.956		

The two rare but critical controller decisions — interruption and backchannel recognition — stay near-perfect (F1 0.97/0.99), and the new promptable-controller tags are perfect on the held-out data. The small accuracy drop versus v5 (0.975) is the price of the instruction-conditioned training and the distill term.

C. Controller probes

We additionally probe the runtime query path directly (guards and thresholds applied), on hand-built conversations: user speaking (mid-question) $\rightarrow$ speaking; finished question + silence $\rightarrow$ finish (fires the answer); assistant mid-utterance + user barge-in $\rightarrow$ interruption; partial barge-in $\rightarrow$ interruption; “aham” during assistant speech $\rightarrow$ backchannel (not an interruption); silence during assistant speech $\rightarrow$ keep talking; all correct with probability ≥ 0.83 at the decision point, with the remaining ones carried by the no-voice guard / the state machine.

Promptable-controller probes (rule rendered as the system item): CPF rule + letter in the dictation rhythm $\rightarrow$ take_floor (1.00); the same rule with digit chunks only $\rightarrow$ no fire; *no rule* + letter $\rightarrow$ no fire (the system tags never fire without an instruction); handover rule + trigger $\rightarrow$ handover (1.00); rule + unrelated words $\rightarrow$ no fire. In the live demo, the rule detection uses STT digits normalized to words and per-word chunking; the letter fires at 0.60–0.86 once the dictation rhythm is established.

D. End-to-end

In the browser demo (a Banco Penha banking assistant persona), a stock Qwen3-4B answers “tudo bem?”, is asked for a credit card and replies by asking for the CPF (with conversation history, it never re-asks), and is interrupted when the user speaks over it. With the prompt rule configured, dictating “1, 2, 3, x, 5” makes the companion fire `system_take_floor` on the letter: the system instantly cuts the user off with “Apenas números são aceitos no CPF.” while the companion’s decisions drive the entire exchange; the big model never sees a special token.

V. DISCUSSION AND LIMITATIONS

a) Why not fine-tune the big model.: The whole point is composability: the companion is a 0.8B sidecar trained in hours on one GPU; swapping the big model (a better checkpoint, a GGUF, a different family) requires no retraining of the turn-taking skill.

b) Turn-end detection.: The learned turn-end signal is sentence-final punctuation of the final chunk plus silence. The ASR must therefore punctuate (faster-whisper does by default); unpunctuated transcripts delay the “finish” decision until the next word batch or a second silence.

c) Qwen3.5-4B as the big model.: The user’s target big model, Qwen3.5-4B, verbalizes its chain of thought (“thinking / Thinking Process:” text) even with thinking disabled in the chat template and strong system directives — the same behavior that led the sft project to reject it. The demo therefore defaults to Qwen3-4B-Instruct-2507; the companion is model-agnostic and the 4B remains selectable (e.g., served by llama.cpp).

d) Barge-in during a closed sentence.: Barge-ins over an *open* assistant item are detected with probability 1.0; barge-ins over a closed sentence are detected with ~ 0.8 probability. The

orchestrator stops the answer on any user words regardless, so the system behaves correctly even when the tag is marginal.

e) Data quality lesson.: Two bugs in the data pipeline were found by eval-driven debugging (a shared token cache corrupting input text across dialogues, and a duplicated chunk in silence records); fixing them moved held-out accuracy from 0.70 to 0.98. We recommend always verifying prep output against the source dialogues.

f) Prompt-rule detection vs. hardcoded words.: The rule mechanism is learned, not coded: the held-out data shows perfect precision/recall on the system tags, and probes confirm the model generalizes the letter rule to unseen letters (a–z) and number words. The one remaining weakness is rhythm-dependent: a letter that follows just a single digit chunk is a weak signal (0.17–0.60), while the same letter after two or more digit chunks fires reliably (0.60–0.86). A data variant with more digit-count diversity is the planned fix.

g) System training cost on constrained hardware.: The companion training must run within 16 GB VRAM under WSL2, where the dxg driver mirrors GPU memory into host RAM: a per-device batch of 2 (the original setting) deterministically OOMs the 31 GB VM (15.8 GB VRAM $\rightarrow$ ~ 31 GB host anon-RSS). Batch 1×16 accumulation keeps VRAM at ~ 11 GB at the same effective batch, at the cost of ~ 5 h wall time.

VI. CONCLUSION

We presented MTP-Duplex: attach full-duplex turn-taking to any LLM by training a small protocol companion that predicts the DuplexCascade control stream on behalf of the big model. On the DuplexCascade pt-BR dataset the companion reaches 95.6% tag accuracy with near-perfect interruption/backchannel recognition, and its *promptable controller* extends the protocol with instruction-conditioned system tags (take-the-floor and handover) learned under a distill objective that preserves instruction-following. The companion successfully drives a stock big model in a live full-duplex browser demo — banking assistant flow with instant rule interrupts, no VAD, no big-model fine-tuning. The approach decouples answer quality from turn-taking skill and makes the duplex protocol a reusable, attachable module.

REFERENCES

- [1] J. Yang, Y. Fujita, and Y. Sudo, “DuplexCascade: full-duplex speech-to-speech dialogue with VAD-free cascaded ASR–LLM–TTS pipeline and micro-turn optimization,” 2026, arXiv:2603.09180. Demo: sbintutions.github.io/DuplexCascadeDemo.
- [2] DeepSeek-AI, “DeepSeek-V3 technical report,” 2024, arXiv:2412.19437.
- [3] Qwen Team, “Qwen3.5: towards native multimodal agents,” Feb. 2026, qwen.ai/blog?id=qwen3.5.
- [4] Unsloth AI, “Unsloth: faster fine-tuning,” unsloth.ai.